# Building an Agent Harness Framework Using Claude: Architecture, Patterns, and Comparative Analysis

## 1 Executive Summary

A production-grade agent harness built on Claude can match or exceed the performance of leading single-model systems by adopting the multi-agent orchestration architecture proven by Microsoft's MDASH, which scored 88.45% on CyberGym—over five points ahead of Anthropic's Mythos Preview (83.1%) and GPT-5.5 (81.8%) [1]. This gap demonstrates that the harness itself, not just the underlying model, is the decisive factor in system performance for complex agentic tasks.

The most effective architecture follows MDASH's three-stage generator-verifier-prover pipeline: specialized agents detect potential findings, debate agents challenge validity to reduce false positives, and proof-of-concept agents confirm results [1]. Claude's current capabilities—tool use, extended thinking with configurable deliberation budgets, the Agent SDK's durable session management, and integration with Microsoft's Agent Framework via the BaseAgent interface—provide all the building blocks needed to implement this pattern [2, 3, 4, 5]. Extended thinking is particularly well-suited for the debate stage, where agents must reason deeply about whether findings are genuine.

The critical implementation insight is treating the harness as a distributed systems problem: durable execution with external state persistence ensures no progress is lost, circuit breakers and idempotent tool calls prevent cascading failures, and explicit contracts between agents enable model-swapping without rewriting infrastructure [6, 7]. Practitioners should begin with container-based sandboxed deployments, implement DAG-based parallel execution for independent subtasks, and layer in OpenTelemetry observability for cost tracking and debugging [4, 8, 9]. This approach transforms Claude from a capable single model into a coordinated multi-agent system that leverages architectural advantages rather than relying solely on model scale.

## 2 Introduction: The Case for Multi-Agent Harnesses

The emergence of multi-agent orchestration systems represents a paradigm shift in how AI tackles complex, real-world tasks. Microsoft's Multi-Model Agentic Scanning Harness (MDASH) scored 88.45% on the CyberGym benchmark, surpassing Anthropic's Mythos Preview at 83.1% and GPT-5.5 at 81.8% [1]. This result demonstrates a critical insight for practitioners building agent systems: multi-agent orchestration patterns can surpass even the most capable single models for complex agentic workflows [1]. MDASH orchestrates more than 100 specialized agents across an ensemble of frontier and custom models [10], proving that the harness architecture itself—not just the underlying model—is a decisive factor in system performance.

This report provides a comprehensive blueprint for building a production-grade agent harness using Claude, drawing architectural inspiration from Microsoft's MDASH, Anthropic's Mythos scaffolding patterns, and established multi-agent coordination frameworks. The goal is to equip practitioners with the design principles, coordination patterns, and implementation strategies needed to construct a harness that rivals state-of-the-art systems.

## 3 Microsoft MDASH Architecture Deep Dive

**The Staged Pipeline Design.** MDASH's architecture implements a generator-verifier-prover pattern through a staged pipeline. First, scanning agents examine code for potential vulnerabilities; then a separate set of agents "debate" whether each finding is real and exploitable; finally, a proof-of-concept stage constructs attacks to confirm bugs exist [1]. This three-stage approach ensures that each phase applies specialized reasoning—detection, verification, and exploitation—rather than asking a single model to handle all aspects simultaneously.

**Multi-Model Ensemble Orchestration.** Unlike single-model competitors, MDASH orchestrates more than 100 specialized AI agents across an ensemble of frontier and distilled models to find, debate, and prove exploitable bugs from end to end [10]. This ensemble approach allows different models to be optimized for different subtasks: some agents may excel at pattern recognition in code, while others specialize in adversarial reasoning or exploit construction. The result is a system that achieves an industry-leading 88.45% score on CyberGym—roughly five points ahead of the next entry on the leaderboard [11].

**Real-World Validation.** Beyond benchmark performance, MDASH discovered 16 previously unknown Windows vulnerabilities, including four critical remote code execution flaws in the Windows networking and authentication stack, which were fixed in the May 2026 Patch Tuesday [12]. This demonstrates that the multi-agent harness architecture translates directly into practical security value, not merely academic benchmark improvements.

## 4 Anthropic Mythos and Single-Model Baselines

**What Mythos Is.** Claude Mythos Preview is, by every published benchmark, the most capable single AI model ever built, scoring 93.9% on SWE-bench Verified, 97.6% on the USAMO math olympiad, and 83.1% on CyberGym [13]. Critically, it is a single model running inside an agent framework, not a multi-agent system [1]. Anthropic deemed it too dangerous for public release, instead providing access through Project Glasswing to approximately 40 organizations including Amazon, Apple, Google, Microsoft, and CrowdStrike, with $100 million in usage credits [13].

**Autonomous Task Execution Capabilities.** Researchers developed scaffolds allowing Mythos to turn vulnerabilities into exploits without any human intervention. Engineers at Anthropic with no formal security training asked Mythos Preview to find remote code execution vulnerabilities overnight and woke up the following morning to complete, working exploits [14]. This demonstrates that even a single powerful model, when wrapped in appropriate scaffolding, can achieve remarkable autonomous task execution.

**The Multi-Agent Advantage.** Despite Mythos' s extraordinary single-model capabilities, Microsoft' s MDASH multi-agent harness outperformed it by over 5 percentage points on CyberGym [1]. This gap illustrates a fundamental architectural lesson: for tasks requiring diverse reasoning modes—detection, verification, adversarial thinking—a well-orchestrated ensemble of specialized agents can exceed what even the most capable monolithic model achieves alone.

## 5 Core Agent Harness Architecture Patterns

**The Harness as Infrastructure Layer.** The agent harness is "the layer where model reasoning connects to real execution: shell and filesystem access, approval flows, and context management across long-running sessions" [15]. The model generates responses; the harness handles everything else—tool orchestration, filesystem access, human approvals, sub-agent coordination, state management, and failure recovery [16]. This separation of concerns is the foundational design principle that enables stable, evolvable systems.

**Durable Execution as the Production Primitive.** Durable execution has emerged as the critical infrastructure pattern for agent harnesses, crossing into the early majority in 2025 with offerings from AWS, Cloudflare, and Vercel, driven primarily by AI agent infrastructure needs [17]. Microsoft defines durable execution through three core principles: incremental execution where each operation runs independently and in order, state persistence where the output of each step is durably saved to ensure progress is not lost, and fault tolerance where if a step fails the operation is retried from the last successful step [6].

**Fault Tolerance from Distributed Systems.** Agentic AI requires the same operational playbook as microservices: timeouts, retries with backoff, circuit breakers to avoid cascading failures, and fallback behaviors that let a workflow degrade gracefully. For tool calls, it also helps to design for idempotency so retries do not produce side effects [7]. A critical design principle is that state must be external—storing workflow state inside the prompt guarantees data loss and audit failures [18].

**Decoupling Model Logic from Infrastructure.** Defining explicit contracts (structured inputs/outputs with JSON schema) and tool allowlists enables swapping models, changing retrieval methods, or adding workflow steps without rewriting the system [7]. This principle is essential for building a harness that can evolve as models improve—the same harness infrastructure should work whether the underlying model is Claude, GPT, or a specialized fine-tuned model.

## 6 Multi-Agent Coordination Patterns

**Orchestration Taxonomy.** Semantic Kernel now supports several orchestration patterns that map directly to harness design: Sequential Orchestration (pipeline where each agent processes output from the previous), Concurrent Orchestration (multiple agents work in parallel with results aggregated), Group Chat Orchestration (collaborative conversation among agents for debates or problem-solving), Handoff Orchestration (agents transfer control based on context), and Magentic Orchestration (flexible general-purpose pattern where a dedicated manager coordinates specialized agents) [19].

**Generator-Verifier with DAG-Based Execution.** The Verified Multi-Agent Orchestration (VMAO) framework demonstrates an advanced coordination pattern with dependency-aware parallel execution over a DAG of subquestions with automatic context propagation and verification-driven adaptive replanning that uses an LLM-based verifier as an orchestration-level coordination signal [9]. This pattern directly mirrors MDASH' s approach where scanning agents generate findings, debate agents verify them, and proof-of-concept agents confirm exploitability.

**Debate and Reflection Mechanisms.** Multi-agent AI systems can leverage patterns including group-chat, reflection, selector, and swarm orchestration [20]. The debate mechanism—where agents argue for and against a finding' s validity—is particularly powerful for reducing false positives in vulnerability detection and ensuring high-

confidence outputs. MDASH's debate stage exemplifies this: separate agents challenge each finding's validity before resources are spent on proof-of-concept generation [1].

## 7 Building the Harness with Claude

**Claude's Agentic Building Blocks.** Claude provides several capabilities essential for harness construction. Tool use lets Claude call developer-defined functions, deciding when to invoke them based on user requests and tool descriptions, then returning structured calls that applications execute [2]. Advanced tool use supports IDE assistants integrating git operations, file manipulation, testing frameworks, and operations coordinators connecting Slack, GitHub, Google Drive, Jira, and dozens of MCP servers simultaneously [21].

**Extended Thinking for Deep Deliberation.** Claude 4 models support extended thinking, which allows Claude to alternate between reasoning and tool use during deliberation, providing transparency into step-by-step decision-making [3]. Developers can explicitly allocate deliberation budget to control reasoning depth [22]. This capability is critical for implementing the "debate" stage of an MDASH-like pipeline, where agents need to reason deeply about whether a finding is genuine.

**The Claude Agent SDK for Production Harnesses.** The Claude Agent SDK maintains conversational state and executes commands in a persistent environment, supporting multi-turn conversations with automatic session resumption, streaming responses in real-time, and structured output from agents [4]. The SDK provides hooks for intercepting and controlling agent behavior, checkpointing to rewind file changes, cost/usage tracking, and OpenTelemetry-based observability [8].

**Deployment Patterns.** For production hosting, Anthropic recommends container-based sandboxing with three patterns: ephemeral sessions (create a new container for each user task, then destroy it when complete), long-running sessions (maintain persistent container instances for long-running tasks, often running multiple Claude Agent processes inside the container), and hybrid sessions [4]. Microsoft's Durable Task framework for AI agents enables resilient, stateful agentic workflows using standard programming constructs while the runtime persists state and recovers from failures automatically [23].

**Multi-Agent Integration via BaseAgent Interface.** The Microsoft Agent Framework integration provides consistent agent abstraction where Claude agents implement the same BaseAgent interface as every other agent type in the framework, enabling multi-agent workflows in sequential, concurrent, handoff, and group chat patterns using built-in orchestrators [5]. The framework converges the best of Semantic Kernel and AutoGen into a single SDK to define agents, tools, and explicit workflows for multi-step tasks and human-in-the-loop scenarios [24].

## 8 Benchmark Comparison and Validation Strategy

Understanding how different systems perform across benchmarks informs harness design decisions. The following table summarizes the current state of the art:

| Benchmark | What It Measures | MDASH | Mythos | GPT-5.5 |
|---|---|---|---|---|
| CyberGym (1,507 tasks) | Vulnerability reproduction via PoC generation [25] | **88.45%** [1] | 83.1% [13] | 81.8% [1] |
| SWE-bench Pro | Complex software engineering tasks [26] | N/A | **77.8%** [27] | 58.6–64.3% [27] |
| Gaia2 | Multi-step reasoning with dynamic environments [28] | N/A | — | 42% (GPT-5) [28] |

Table: Benchmark Performance Comparison Across Leading Agent Systems

Chart Explanation: This chart visualizes the performance gap between Microsoft's multi-agent MDASH system (88.45%) and single-model approaches from Anthropic Mythos (83.1%) and OpenAI GPT-5.5 (81.8%) on the CyberGym benchmark [1, 29, 11]. The 5.35-percentage-point gap between MDASH and Mythos demonstrates the advantage of multi-agent orchestration over even the most capable individual models.

CyberGym was created by UC Berkeley researchers and tasks agents with generating a proof-of-concept test that reproduces a vulnerability given its text description and codebase [25]. Even top-performing combinations initially achieved only approximately 20% success rate [30], making MDASH's 88.45% score a remarkable achievement. Notably, under multi-task loads, leading computer-using agents degrade sharply on web tasks, with completion rates dropping from 16.7% to 8.7% [31], underscoring the importance of robust harness architecture for maintaining performance under load.

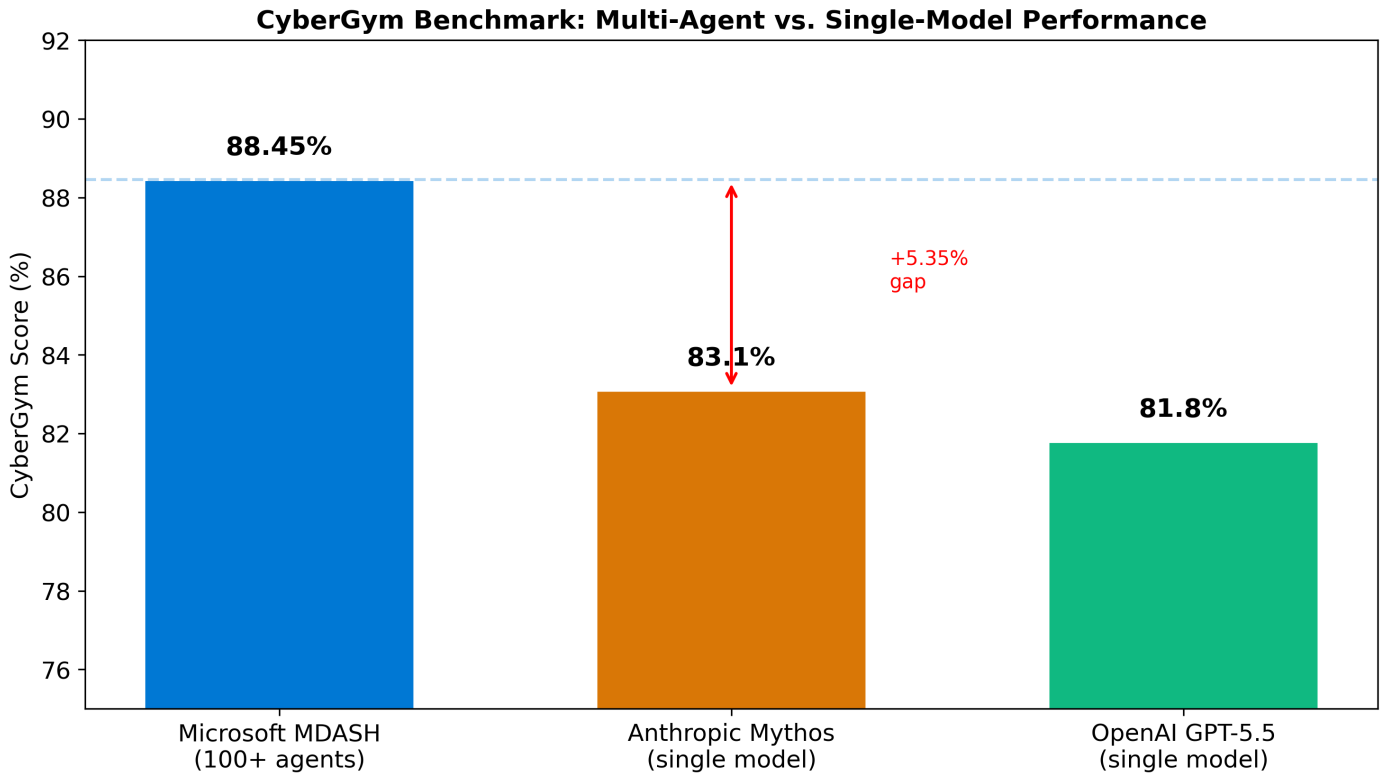


Figure 1: CyberGym Benchmark Comparison

## 9 Implementation Blueprint for a Claude-Based Agent Harness

Based on the architectural patterns from MDASH, Mythos scaffolding, and production harness best practices, the following blueprint outlines how to construct a comparable system using Claude.

**Stage 1: Foundation Layer.** Implement durable execution with external state storage, ensuring every agent step is persisted and recoverable [6, 18]. Use the Claude Agent SDK's session management for maintaining conversational state across long-running tasks [4], and deploy within container-based sandboxes following the hybrid session pattern for flexibility [4].

**Stage 2: Specialized Agent Pipeline.** Mirror MDASH's three-stage architecture by creating specialized Claude agents for distinct phases—analysis, verification, and proof generation [1]. Each agent should have explicit contracts defining its inputs, outputs, and tool allowlists [7]. Leverage extended thinking with configurable deliberation budgets for agents that require deep reasoning [3, 22].

**Stage 3: Multi-Agent Orchestration.** Implement the generator-verifier pattern using Semantic Kernel's orchestration primitives [19] or the Microsoft Agent Framework's BaseAgent interface [5]. Include debate mechanisms where verification agents challenge findings before escalation, and use DAG-based parallel execution for independent subtasks [9].

**Stage 4: Reliability and Observability.** Apply distributed systems patterns: circuit breakers, retries with exponential backoff, idempotent tool calls, and graceful degradation [7]. Integrate OpenTelemetry-based observability from the Claude Agent SDK for monitoring agent behavior, tracking costs, and debugging failures [8].

## 10 Conclusion

The evidence from MDASH's benchmark-leading performance demonstrates that multi-agent harness architecture is a decisive competitive advantage over single-model approaches, even when those single models are as capable as Anthropic's Mythos [1]. The key architectural principles—staged pipelines with generator-verifier-prover patterns, durable execution with external state, fault-tolerant orchestration, and explicit contracts for model-infrastructure decoupling—are all implementable using Claude's current capabilities including tool use, extended thinking, the Agent SDK, and integration with Microsoft's Agent Framework [2, 3, 4, 5].

Building such a system requires treating the harness as a first-class engineering artifact: the infrastructure that connects model reasoning to real execution determines system reliability, scalability, and ultimately performance

[15, 16]. By combining Claude's strong single-model capabilities with multi-agent orchestration patterns proven by MDASH, practitioners can build systems that approach or exceed the performance of any individual model operating alone.

## 11 References


1. https://www.geekwire.com/2026/microsofts-multi-agent-ai-system-tops-anthropics-mythos-on-cybersecurity-benchmark/

2. https://docs.anthropic.com/en/docs/agents-and-tools/

3. https://www.anthropic.com/news/claude-4

4. https://code.claude.com/docs/en/agent-sdk/hosting

5. https://devblogs.microsoft.com/semantic-kernel/build-ai-agents-with-claude-agent-sdk-and-microsoft-agent-framework

6. https:// techcommunity.microsoft.com/ blog/ appsonazureblog/ building-durable-and-deterministic-multi-agent-orchestrations-with-durable-execu/4408842

7. https://developers.redhat.com/articles/2026/02/11/agentic-ai-design-reliable-workflows-across-hybrid-cloud

8. https://code.claude.com/docs/en/agent-sdk/python

9. https://arxiv.org/html/2603.11445v1

10. https:// siliconangle.com/ 2026/05/13/ microsofts-agentic-security-system-mdash-uncovers-four-critical-windows-rce-flaws/

11. https:// www.thurrott.com/ microsoft/ 336046/ now-microsoft-has-an-agentic-ai-system-for-finding-security-vulnerabilities-too

12. https://thehackernews.com/2026/05/microsofts-mdash-ai-system-finds-16.html?

13. https:// www.forbes.com/ sites/ jonmarkman/ 2026/04/08/ what-is-claude-mythos-and-why-anthropic-wont-let-anyone-use-it/

14. https://red.anthropic.com/2026/mythos-preview/

15. https://devblogs.microsoft.com/agent-framework/agent-harness-in-agent-framework/

16. http://metavert.io/agent-harness

17. https://www.inngest.com/blog/durable-execution-key-to-harnessing-ai-agents

18. https://spiralscout.com/blog/agentic-ai-architecture-production-patterns

19. https://devblogs.microsoft.com/semantic-kernel/semantic-kernel-multi-agent-orchestration/

20. https://developer.microsoft.com/en-us/reactor/events/25313/

21. https://www.anthropic.com/engineering/advanced-tool-use

22. https://support.anthropic.com/en/articles/10574485-using-extended-thinking

23. https://learn.microsoft.com/en-us/azure/durable-task/sdks/durable-task-for-ai-agents

24. https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/empowering-multi-agent-solutions-with-microsoft-agent-framework—code-migration/4468094

25. https://arxiv.org/html/2506.02548v2

26. https://scale.com/leaderboard/swe_bench_pro_public

27. https:// www.rdworldonline.com/ how-openais-recently-released-gpt-5-5-stacks-up-with-anthropics-gated-claude-mythos/

28. https://openreview.net/forum?id=9gw03JpKK4

29. https:// www.pymnts.com/ cybersecurity/ 2026/ microsoft-beats-anthropic-and-openai-on-key-cybersecurity-test/

30. http://arxiv.org/abs/2506.02548v2

31. https://www.microsoft.com/en-us/research/blog/corpgen-advances-ai-agents-for-real-work/